

Dynamic Programming (II)

- ▷ Matrix-Chain Multiplication
- ▷ 0/1 Knapsack

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

Recipe for Applying Dynamic Programming

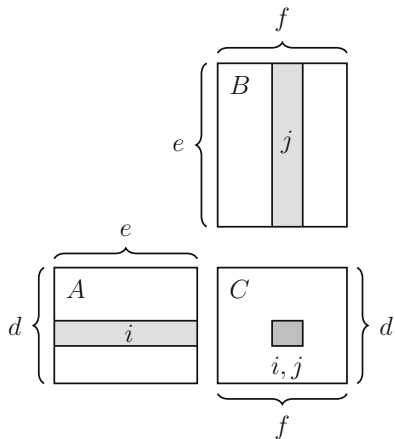
- **Step 1:** Identify **optimal substructure**.
- **Step 2:** Find a **recursive formulation** for the value of an optimal solution.
- **Step 3:** **Compute the optimal value** using dynamic programming.
- **Step 4:** Keep track of the choices, if needed, to **construct an optimal solution** from the computed information.

Matrix Chain Multiplication

- Review: matrix multiplication.

- $C = A * B$.
- A is $d \times e$ and B is $e \times f$.
- $d \cdot e \cdot f$ scalar multiplications.

$$C[i, j] = \sum_{k=0}^{e-1} A[i, k] B[k, j].$$



Matrix Chain Multiplication

- **Goal:** Compute $A = A_0 * A_1 * \dots * A_{n-1}$ using the **minimum number of scalar multiplications**.
 - A_i has dimensions $d_i \times d_{i+1}$.
- We use **parenthesization** to specify how to compute the matrix product.
 - Different parenthesizations give different orders of multiplication.
- Example: B is 3×100 , C is 100×5 , and D is 5×5 .

$$(B * C) * D : \quad 3 \cdot 100 \cdot 5 + 3 \cdot 5 \cdot 5 = \mathbf{1575},$$

$$B * (C * D) : \quad 100 \cdot 5 \cdot 5 + 3 \cdot 100 \cdot 5 = 4000.$$

Both parenthesizations give the same matrix, but their costs differ.

How should we parenthesize the product to minimize its cost?

Enumeration Approach for MCM

- **Matrix-chain multiplication by enumeration:**
 - Try all ways to parenthesize $A_0 * A_1 * \dots * A_{n-1}$.
 - Calculate the number of operations for each one.
 - Pick the parenthesization with the smallest cost.
- **Running time:**
 - A parenthesization corresponds to a full binary expression tree with n leaves and $n - 1$ internal nodes.
 - The number of parenthesizations is a Catalan number and grows exponentially with n .

Trying every parenthesization takes exponential time.

Greedy Approach for MCM

- Idea #1: repeatedly choose the adjacent product that uses the fewest operations.
- Counterexample:

$$A : 101 \times 11, \quad B : 11 \times 9, \quad C : 9 \times 100, \quad D : 100 \times 99.$$

Greedy gives $A * ((B * C) * D)$, with cost

$$9900 + 108900 + 109989 = \mathbf{228789}.$$

But $(A * B) * (C * D)$ costs

$$9999 + 89100 + 89991 = \mathbf{189090}.$$

The greedy choice does not always give an optimal parenthesization.

Choosing the Final Multiplication

- Any parenthesization of $A_0 * \dots * A_{n-1}$, for $n \geq 2$, has a **final multiplication**.
- If the final split is after A_k , it combines these two subchain products:

$$\underbrace{(A_0 * \dots * A_k)}_{d_0 \times d_{k+1}} * \underbrace{(A_{k+1} * \dots * A_{n-1})}_{d_{k+1} \times d_n}.$$

- The final multiplication costs $d_0 d_{k+1} d_n$ scalar multiplications.
 - The total cost also includes computing both subchain products.
- There are $n - 1$ possible final splits: $k = 0, 1, \dots, n - 2$.

Optimal Substructure of Matrix Chains

Suppose an **optimal parenthesization** of $A_0 * \dots * A_{n-1}$ splits after A_k :

$$(A_0 * \dots * A_k) * (A_{k+1} * \dots * A_{n-1}).$$

- Within this solution, both subchains must have **optimal parenthesizations**.
- Otherwise, replace one subchain's parenthesization with a cheaper one.
 - Its matrix product is unchanged, so the other subchain and the final multiplication remain unchanged.
 - The total cost would decrease, contradicting optimality.

Use optimal subchain costs for each split, then choose the smallest total cost.

Dynamic Programming Approach for MCM

Let $N_{i,j}$ denote the minimum cost of computing $A_i * \dots * A_j$.

For $n \geq 2$, the minimum cost for the whole chain satisfies

$$N_{0,n-1} = \min_{0 \leq k < n-1} \left\{ N_{0,k} + N_{k+1,n-1} + d_0 d_{k+1} d_n \right\}.$$

For a minimizing split k , the optimal value is the sum of:

- the optimal costs of the two subchains;
- the cost of their final multiplication.

Define Subproblems for MCM

- For $0 \leq i \leq j < n$, define the subproblem as finding a minimum-cost parenthesization of the subchain

$$A_i * A_{i+1} * \cdots * A_j.$$

- Let $N_{i,j}$ denote its **minimum number of scalar multiplications**.
 - $N_{i,i} = 0$: one matrix requires no multiplication.
- The optimal value for the whole problem is $N_{0,n-1}$.

Recurrence of Matrix-Chain Multiplication

For $i < j$, the recurrence is

$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}.$$

- We have **overlapping subproblems**: different recursive calls can solve the same subproblem. Saving their answers avoids repeated computation.
- For example, the recursive computations of $N_{2,6}$ and $N_{3,7}$ both include

$$N_{3,6}, \quad N_{4,6}, \quad N_{5,6}, \quad N_{6,6}.$$

Recursive Approach

Input: chain S with dimensions $d_0, \dots, d_n$; indices i, j .

Output: minimum cost for $A_i * \dots * A_j$.

procedure RecursiveMatrixChain(S, i, j)

if $i = j$ **then return** 0

$N_{i,j} \leftarrow \infty$

for $k \leftarrow i$ **to** $j - 1$ **do**

$q \leftarrow \text{RecursiveMatrixChain}(S, i, k)$

$+ \text{RecursiveMatrixChain}(S, k + 1, j) + d_i d_{k+1} d_{j+1}$

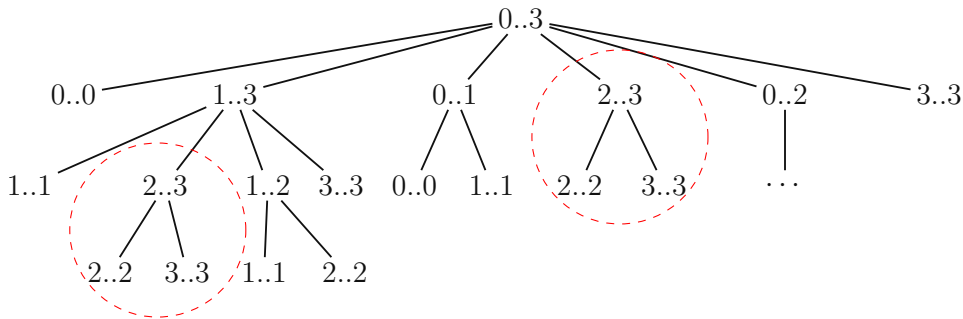
$N_{i,j} \leftarrow \min\{N_{i,j}, q\}$

return $N_{i,j}$

We can use **memoization** to avoid repeated computations (details omitted).

Next, we focus on the **bottom-up DP implementation**.

Subproblem Overlap



How should we schedule the subproblems?

Dynamic Programming Algorithm

- Optimal substructure and overlapping subproblems suggest a dynamic programming approach.
- Compute optimal values **bottom-up** and remember them.
- Start with the base cases: $N_{i,i} = 0$.
- Then solve chains of length $2, 3, \dots, n$.
- Running time: $O(n^3)$.

Dynamic Programming Algorithm

Input: chain S with dimensions $d_0, \dots, d_n$.

Output: the minimum cost of parenthesizing S .

procedure MatrixChain(S)

for $i \leftarrow 0$ **to** $n - 1$ **do**

$N_{i,i} \leftarrow 0$

for $b \leftarrow 1$ **to** $n - 1$ **do**

for $i \leftarrow 0$ **to** $n - b - 1$ **do**

$j \leftarrow i + b$

$N_{i,j} \leftarrow \infty$

for $k \leftarrow i$ **to** $j - 1$ **do**

$q \leftarrow N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}$

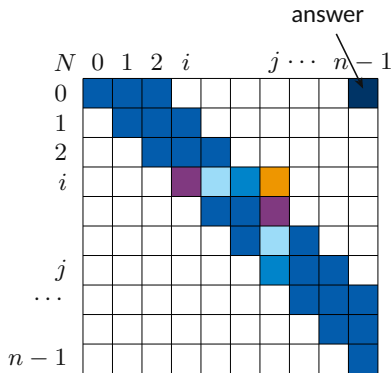
$N_{i,j} \leftarrow \min\{N_{i,j}, q\}$

return $N_{0,n-1}$

▷ $b = j - i$; chain length = $b + 1$

Algorithm Visualization

- Fill N by diagonals: shorter chains first.
- $N_{i,j}$ uses earlier entries in row i and column j .
- Each entry takes $O(n)$ time.
- $\Theta(n^2)$ entries give an $O(n^3)$ time bound.
- For each entry $N_{i,j}$ with $i < j$, record a minimizing split k to recover the parenthesization.



$$N_{i,j} = \min_{i \leq k < j} \{N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}.$$

$$A_0 : 30 \times 35, \quad A_1 : 35 \times 15, \quad A_2 : 15 \times 5,$$

$$A_3 : 5 \times 10, \quad A_4 : 10 \times 20, \quad A_5 : 20 \times 25.$$

0	1	2	3	4	5	
0	15 750	7 875	9 375	11 875	15 125	0
	0	2 625	4 375	7 125	10 500	1
		0	750	2 500	5 375	2
			0	1 000	3 500	3
				0	5 000	4
					0	5

To compute $N_{1,4}$, try $k = 1, 2, 3$:

$$k = 1 : \quad N_{1,1} + N_{2,4} + d_1 d_2 d_5$$

$$= 0 + 2500 + 35 \cdot 15 \cdot 20 = 13000,$$

$$k = 2 : \quad N_{1,2} + N_{3,4} + d_1 d_3 d_5$$

$$= 2625 + 1000 + 35 \cdot 5 \cdot 20 = \mathbf{7125},$$

$$k = 3 : \quad N_{1,3} + N_{4,4} + d_1 d_4 d_5$$

$$= 4375 + 0 + 35 \cdot 10 \cdot 20 = 11375.$$

$N_{1,4} = 7125$, with split $k = 2$.

$$(A_0 * (A_1 * A_2)) * ((A_3 * A_4) * A_5).$$

Optimal costs $N_{i,j}$

0	1	2	3	4	5	
0	15 750	7 875	9 375	11 875	15 125	0
	0	2 625	4 375	7 125	10 500	1
		0	750	2 500	5 375	2
			0	1 000	3 500	3
				0	5 000	4
					0	5

Minimizing splits $K_{i,j}$

0	1	2	3	4	5	
	0	0	2	2	2	0
		1	2	2	2	1
			2	2	2	2
				3	4	3
					4	4
						5

Recovering the Parenthesization

When computing $N_{i,j}$, record a split attaining the minimum in $K_{i,j}$. The right-hand table on the previous slide contains these split positions.

```
procedure Print-Parens( $K, i, j$ )  
  if  $i = j$  then  
    output  $A_i$   
  else  
     $k \leftarrow K_{i,j}$   
    output (  
      Print-Parens( $K, i, k$ )  
      Print-Parens( $K, k + 1, j$ )  
    output )
```

Call Print-Parens($K, 0, n - 1$). Reconstruction takes $O(n)$ time.

Matrix Chain Multiplication

- Some final thoughts:
 - We replaced an exponential algorithm with a $\Theta(n^3)$ algorithm.
 - Naive recursion makes repeated calls, but there are only $\Theta(n^2)$ distinct subproblems.
 - Once the final split is fixed, the two subchains can be optimized independently.

Matrix Chain Multiplication Summary

- Determine the cost of each adjacent pair of matrices.
 - Then find the minimum cost of each chain of three matrices, using the precomputed costs of shorter chains.
 - Each three-matrix chain has two possible final splits.
- Continue until we find the minimum cost of all n matrices.
 - Use the optimal costs of shorter consecutive subchains.
 - For the whole chain, there are $n - 1$ possible final splits.

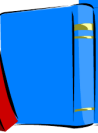
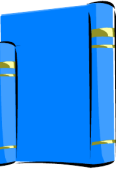
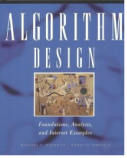
The 0/1 Knapsack Problem

- Given: a set S of n items, with item i having
 - w_i : a positive integer weight;
 - b_i : a positive benefit.
- Goal: choose items with maximum total benefit and total weight at most W , a nonnegative integer capacity.
- Each item is taken whole or left out; fractional amounts are not allowed.
- Let $T \subseteq S$ be the set of items we take.

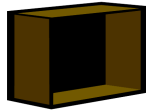
$$\text{maximize } \sum_{i \in T} b_i \quad \text{subject to } \sum_{i \in T} w_i \leq W.$$

Example

Given items with benefits b_i and weights w_i , choose a subset with maximum benefit and total weight at most W .

Items:					
	1	2	3	4	5
Weight:	4 in	2 in	2 in	6 in	2 in
Benefit:	\$20	\$3	\$6	\$25	\$80

“knansack”



box of width 9 in

Solution:

- item 5 (\$80, 2 in)
- item 3 (\$6, 2 in)
- item 1 (\$20, 4 in)

First Attempt

- S_k : the set of items numbered 1 to k .
- First attempt: $B[k]$ stores the best selection from S_k for capacity W .
- Problem: **optimal substructure** does not hold for these subproblems.

For capacity $W = 20$, consider these (benefit, weight) pairs:

$$S = \{(3, 2), (5, 4), (8, 5), (4, 3), (10, 9)\}.$$

Best for S_4 :

(3, 2)	(5, 4)	(8, 5)	(4, 3)	
--------	--------	--------	--------	--

Best for S_5 :

(3, 2)	(5, 4)	(8, 5)	(10, 9)
--------	--------	--------	---------

← 20 →

The optimal solution for S_5 does not contain the optimal solution for S_4 .

Optimal Substructure of 0/1 Knapsack

Consider an optimal selection from $S_k = \{1, \dots, k\}$ with capacity w .

- If it **skips item k** , it must be optimal for S_{k-1} with capacity w .
- If it **takes item k** , its remaining selection must be optimal for S_{k-1} with capacity $w - w_k$.
- Otherwise, replace the remaining selection by one of greater benefit within the same remaining capacity. Keeping the decision about item k would improve the total benefit, contradicting optimality.

Both choices remove item k from further consideration.

Second Attempt

Define $B[k, w]$ as the **maximum benefit** from S_k with total weight at most w .

According to the optimal substructure of this problem, we have the recurrence for $k \geq 1$:

$$B[k, w] = \begin{cases} B[k-1, w], & \text{if } w_k > w, \\ \max\{B[k-1, w], B[k-1, w - w_k] + b_k\}, & \text{otherwise.} \end{cases}$$

- Base cases: $B[0, w] = 0$ and $B[k, 0] = 0$.
- The best selection either
 - skips item k , leaving capacity w ; or
 - takes item k , leaving capacity $w - w_k$ and adding benefit b_k .
- Compute rows $k = 1, \dots, n$; both choices use row $k - 1$.

Knapsack Example

Knapsack capacity $W = 5$

Item	Weight	Benefit
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

Items $1, \dots, k$	Capacity w					
	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

$$B[k, w] = \begin{cases} B[k-1, w], & \text{if } w_k > w, \\ \max\{B[k-1, w], B[k-1, w - w_k] + b_k\}, & \text{otherwise.} \end{cases}$$

Knapsack Algorithm

- $B[k, w]$ depends only on row $k - 1$: use two arrays.
 - A holds the previous row.
 - B stores the current row.
- Time: $O(nW)$.
Space for the value: $O(W)$.
- This is **pseudo-polynomial** time: the binary encoding of W has $O(\log(W + 1))$ bits.

Input: weights w_i , benefits b_i , capacity W .

Output: maximum total benefit.

procedure O1Knapsack(S, W)

Let A, B be arrays of length $W + 1$.

for $w \leftarrow 0$ **to** W **do**

$B[w] \leftarrow 0$

for $k \leftarrow 1$ **to** n **do**

Copy array B into array A .

for $w \leftarrow w_k$ **to** W **do**

if $A[w - w_k] + b_k > A[w]$ **then**

$B[w] \leftarrow A[w - w_k] + b_k$

return $B[W]$

Recovering the Selected Items

Retain the full table $B[k, w]$ for this step, using $O(nW)$ space. If $B[k, w] = B[k - 1, w]$, an optimal selection can skip item k .

procedure Recover-Items($B, w_1, \dots, w_n, W$)

$T \leftarrow \emptyset; \quad w \leftarrow W$

for $k \leftarrow n$ **downto** 1 **do**

if $B[k, w] > B[k - 1, w]$ **then**

$T \leftarrow T \cup \{k\}$

$w \leftarrow w - w_k$

return T

Example: $T = \{1, 2, 4\}$, with total weight 5 and total benefit 37.

Each iteration decides one item, so reconstruction takes $O(n)$ time.